TradifyERP

Technical Documentation

Developer Reference Guide

System Version:	TradifyERP v1.0
Total Modules:	12 Modules
Database:	PostgreSQL / SQLite
Framework:	Django 5.2.8
Python Version:	3.14
Generated:	November 22, 2025

Tradify Global
Enterprise Resource Planning System

1. API Reference

TradifyERP provides a comprehensive set of views, models, and utilities for each module. This section documents the key APIs and their usage.

1.1 Accounting Models

```
# Entity Model
class Entity(models.Model):
"""Represents a legal entity or business unit"""
name = models.CharField(max_length=255, unique=True)
code = models.CharField(max_length=50, unique=True)
description = models.TextField(blank=True, null=True)
is_active = models.BooleanField(default=True)

# LedgerEntry Model
class LedgerEntry(models.Model):
"""General Ledger entries"""
account = models.ForeignKey(Account)
entity = models.ForeignKey(Entity)
entry_date = models.DateField()
entry_type = models.CharField(choices=ENTRY_TYPES)
amount = models.DecimalField(max_digits=18, decimal_places=2)
description = models.TextField()
is_posted = models.BooleanField(default=False)
```

1.2 Views & URL Patterns

```
# Dashboard View
class DashboardView(LoginRequiredMixin, TemplateView):
template_name = 'accounting/dashboard.html'

def get_context_data(self, **kwargs):
context = super().get_context_data(**kwargs)
entity = Entity.objects.filter(is_active=True).first()
context['entity'] = entity
context['total_revenue'] = self.calculate_revenue()
return context

# URL Pattern
urlpatterns = [
path('dashboard/', DashboardView.as_view(), name='dashboard'),
path('accounts/', AccountListView.as_view(), name='account_list'),
path('ledger/', LedgerEntryListView.as_view(), name='ledger_list'),
]
```

2. Database Optimization

Indexes: All foreign keys and frequently queried fields have database indexes

Query Optimization: Use select_related() and prefetch_related() for related objects

Caching: Django cache framework for frequently accessed data

Database Pooling: Connection pooling for production deployments

Migrations: All schema changes tracked in migration files

3. Testing Strategy

```
# Unit Test Example
from django.test import TestCase
from accounting.models import Entity, Account

class AccountModelTest(TestCase):
def setUp(self):
self.entity = Entity.objects.create(
name='Test Company',
code='TEST01'
)

def test_account_creation(self):
account = Account.objects.create(
entity=self.entity,
account_number='1000',
name='Cash',
account_type='ASSET'
)
self.assertEqual(account.account_number, '1000')
self.assertTrue(account.is_active)
```